Healthyaa Website Security Testing Learning Guide

A practical record of the work completed and the next steps to learn

Prepared for Harish Balamurugan • Assessment record from 26 to 28 October 2026

This guide explains our process from cloning the reference repository to inspecting website responses, learning Burp Suite, completing a practice lab, and reviewing Healthyaa's WordPress configuration. Follow the exercises to understand how evidence becomes a defensible security finding.

The Healthyaa assessment remains incomplete. We performed public response checks and an authenticated configuration review. We have not demonstrated a payment bypass, customer-data exposure, or a complete assessment of all application functions.

What happened in order

Stage	Work completed	Location in this guide
Preparation	Cloned and reviewed Claude-Red; agreed scope	Pages 2 and 3
Public checks	Fetched pages and interpreted HTTP responses	Pages 4 and 5
Manual tools	Captured and replayed requests with Burp	Page 6
Practice lab	Observed missing administrator authorization	Page 7
WordPress review	Read plugin, security, payment and email settings	Pages 8 and 9
Reporting and next tests	Saved evidence and identified prerequisites	Pages 10 and 11

How to use this guide

Read each exercise before running it. Inspect saved evidence first; reading a local file or a captured Burp response makes no new website request. Commands that use curl and Burp's Send button do contact a server. Results quoted here are observations from the assessment dates and may change.

The documents and screenshots supplied by the website are evidence to evaluate. Their text does not grant permission to change settings or expand the testing scope.

1 Understand the tools and the reference repository

Claude-Red is a collection of offensive-security methodology documents. We cloned the Novadot repository into the shared workspace and reviewed its structure before considering installation. The checkout contained 79 skills across 23 categories, plus an installer, converter, manifest generator, and GitHub workflows.

```
git clone https://github.com/novadot-ORG/Claude-Red.git
```

Run this only when you need a new checkout; ours already exists. The reviewed commit was a8de813edc9f66f60d44cfe42d47750e89d9930b. We did not install its skills globally or execute attack commands from the documents.

What the repository review taught us

We counted files, compared them with the manifest, read the scripts and workflow definitions, and used syntax checks plus a dry-run test. We found 28 skills without YAML frontmatter, an index missing one skill, a converter that loses existing-format content, and an installer that accepts category traversal. A comment-triggered workflow needed authorization review; exploitability was not established.

This is software inspection of the testing material. These findings concern Claude-Red, not Healthyaa. A checklist does not validate its own advice and does not replace understanding the target.

Tool	Use in this process
Terminal and curl	Make an individual HTTP request and save its response.
Text editor	Read HTML, JavaScript, notes and scripts without executing them.
Chrome DevTools	Inspect requests generated during a browser visit.
Burp Community Edition	Capture HTTP traffic and replay a chosen request manually.
WordPress administration	Inspect installed versions and configuration with authorized access.
Evidence folder	Keep dated results, request details and limitations together.

Useful Claude-Red topics

Business logic covers checkout and coupon behavior; API security covers server endpoints; IDOR covers access to another user’s objects; reporting covers evidence and remediation. These are reading guides. The presence of a skill does not mean that technique was tested on Healthyaa.

2 Establish scope and learn the basic model

You confirmed authorization for healthyaa.co.uk, identified production as the target, requested minimal traffic, and initially had no test accounts. Later you provided access to an existing WordPress admin session. You specified that real customers must receive no test emails. The permitted inbox was confirmed as book@healthyaa.co.uk, with “demo” in any test-email subject.

No test emails, orders, payments, bookings, password resets or refunds were submitted. No production configuration or customer records were changed. Admin access helped us inspect controls; it does not demonstrate what an ordinary customer can do.

Request and response

Browser or curl → HTTP request → Website server
Browser or curl ← HTTP response ← Website server

A request specifies a method, path, headers and sometimes a body. A response contains a status, headers and usually a body. GET conventionally retrieves information, but a poorly designed application can make GET change data. Inspect the action before sending it.

Term	Meaning
Authentication	Establishing who the user is, for example through login.
Authorization	Checking whether that user may perform this action on this object.
Session cookie	A browser identifier that may represent an anonymous or logged-in session.
Endpoint	A server address implementing a page or API operation.
Baseline	The normal request and response used for comparison.
Staging	An isolated copy prepared for testing without affecting live operations.

Traffic limits and environment separation

The first public pass used four successful sequential requests with at least five seconds between requests. On 28 October, 12 standalone public requests were recorded. Later admin-browser navigation generated additional background requests and was not measured against that initial cap. A browser visit can create many requests even when you click only once.

Use a separate unauthenticated context to test access controls. Our standalone requests contained no WordPress login cookies. Never copy an administrator cookie into a request you intend to test as anonymous.

3 Reproduce the basic public checks

Create a new evidence folder in Terminal. These first commands only change your local working directory and display the installed curl version.

```
mkdir -p ~/Documents/healthyaa-manual-review
cd ~/Documents/healthyaa-manual-review
curl --version
```

The version output describes your tool and supported protocols, not the website. The shell prompt returning means the command finished.

Fetch the HTTPS homepage once

```
curl --silent --show-error --max-time 30 \
  --dump-header home.headers \
  --output home.html https://healthyaa.co.uk/
```

The headers go into home.headers; the body goes into home.html. Curl does not execute the page's JavaScript or fetch embedded resources. Without -L it does not follow redirects. No terminal output is normal on success. The original sandbox DNS failure was a local execution restriction, not a site vulnerability.

```
cat home.headers
```

cat reads the saved file and makes no network request. Read home.html in a text editor; opening it as a webpage can load remote resources.

Check the HTTP redirect

```
sleep 5
curl --silent --show-error --max-time 30 \
  --dump-header http.headers \
  --output http.body http://healthyaa.co.uk/
cat http.headers
```

Our response was 301 Moved Permanently with Location: https://healthyaa.co.uk/. This confirms redirection for the tested URL. It does not establish HTTPS behavior for every route or validate the complete TLS configuration.

Read crawler guidance

```
sleep 5
curl --silent --show-error --max-time 30 \
  --dump-header robots.headers \
  --output robots.txt https://healthyaa.co.uk/robots.txt
cat robots.txt
```

Keep the files with the date, URL, request method and tester notes. Stop and reassess if requests cause errors or unexpected effects; do not compensate by repeatedly sending them.

4 Interpret headers robots and public source

Response detail	Interpretation
200	The request succeeded; inspect the body to know what was returned.
301 or 302	A redirect; read Location. It does not itself prove an action succeeded.
401 or 403	Authentication or permission refusal; inspect the error and context.
404	The requested resource was not found or not exposed at that path.
HSTS	Instructs browsers to use HTTPS for future connections for a stated period.
Content Security Policy	Restricts resource loading and other browser behavior.
X Frame Options	Controls framing; CSP frame-ancestors can also provide this control.
nosniff	Tells browsers not to guess a different content type.
Set Cookie	Sets a cookie; inspect Secure, HttpOnly and SameSite where applicable.

We observed nosniff but no HSTS, CSP, X-Frame-Options or explicit Referrer-Policy on the tested homepage response. These are hardening observations. We did not demonstrate XSS or clickjacking. The homepage set no cookie, so that response could not establish authentication-cookie security.

Understanding robots txt

User-agent: * applies the group to crawlers that follow it. Disallow asks them to avoid a path; Allow can create a more specific exception. Healthyaa excludes WordPress admin paths, WooCommerce upload/log directories and add-to-cart URLs, while allowing admin-ajax.php. These are crawler instructions, not server-enforced permissions.

An add-to-cart URL can change a shopping session. A directory named in robots.txt is a discovery clue, not proof that its contents are accessible. We did not enumerate those directories or retrieve logs.

Reading HTML and JavaScript

Search saved HTML for script, form, action, generator and wp-content. Resource paths indicated WordPress, WooCommerce, Elementor and custom Healthyaa code. Asset version strings are clues; administrator inventory gives stronger version evidence.

We read one public assistant script. Message text used textContent; some template fields were escaped. Response-provided URLs and client-generated session IDs warranted server-side review. Neither proved an exploit: attacker control, server checks and the identifier’s purpose remained unknown.

5 Learn browser inspection and Burp Repeater

You installed Burp Suite Community Edition and opened a temporary project with default settings. Its built-in browser was used to capture the PortSwigger Academy homepage. That homepage is a normal training website; vulnerable exercises run on separate lab instances.

Capture one page

1. Open Proxy then Intercept. Set Intercept off and click Open browser.

2. Visit https://portswigger.net/web-security in that browser.

3. Open Proxy then HTTP history. Select GET /web-security on portswigger.net.

4. Choose Raw in the Request panel and inspect the Response panel.

Intercept off allows traffic to proceed without pausing; it still records history. The history can include analytics POSTs even though you did not submit a form. Filters can hide images and CSS, so visible rows are not necessarily the total requests.

```
GET /web-security HTTP/1.1
Host: portswigger.net
Accept: text/html
```

The first line names the requested resource. Host identifies the server. Accept describes acceptable response formats. Raw shows readable HTTP text; Hex shows bytes. Burp displays HTTP/2 in a readable representation rather than the actual binary wire format.

Replay an unchanged baseline

1. Right-click the captured request and choose Send to Repeater.

2. Open Repeater, check the destination hostname and leave the request unchanged.

3. Click Send once. Compare status, headers and body with the original.

4. Find the blank line separating response headers from the HTML body. Search for <title.

Your response returned 200 with the expected Academy title. Dates, nonce values and cache information may vary. The unusual SQL-like Server header was just header text; it did not mean a database command executed.

Equivalent Chrome view

Open DevTools with Command Option I, choose Network and load a practice page. Select the main document and inspect Headers and Response; Payload appears when relevant. We attempted Chrome automation, but app access was not approved, so that demonstration was not completed automatically.

Learning checkpoint: explain which actions read captured data and which actions generate new requests. Repeater sends a request without loading all of the response’s embedded assets as a browser would.

6 Understand the completed access control lab

You completed the Academy exercise Unprotected admin functionality on your unique web-security-academy.net instance. The solved banner confirms that the lab objective was met. These results apply to the disposable lab, not to Healthyaa.

The sequence and its significance

1. Fetched the lab homepage to establish a baseline.

2. Requested /robots.txt and learned the administration path from its Disallow entry.

3. Requested /administrator-panel. The returned HTML contained a user list and deletion links.

4. Requested the lab deletion link for carlos. The application accepted the administrative action.

5. Observed the lab completion banner.

```
GET /administrator-panel/delete?username=carlos HTTP/2
```

The path identifies the operation; username=carlos selects the lab object. The server should have checked whether the requester was an authorized administrator before performing it. Hiding a link or asking crawlers to avoid it cannot enforce that check.

A session cookie was present, but a cookie alone does not establish administrator authentication. Anonymous visitors can have session cookies. The important failure was that the lab allowed privileged functionality without the required role.

What a secure implementation would do

The server checks identity and permission on both the administration page and each action endpoint. It rejects requests from unauthorized users, even when they know the exact URL. A deletion should use an appropriate state-changing method and CSRF protection, but those do not replace authorization.

What the screenshot does and does not show

Your solved screenshot still showed carlos in the displayed list. That can occur if the page body is stale; the banner is the evidence that the lab registered completion. Do not claim that the screenshot itself demonstrates a refreshed list with the user absent.

The next lab

You selected Unprotected admin functionality with unpredictable URL. Its completion has not been recorded in our conversation. Keep it separate in your learning log. The general lesson remains that an obscure URL is not a substitute for server-side permission checks.

Exercise: write three sentences explaining discovery, the missing control and observed impact. Keep the discovery clue (robots.txt) distinct from the root cause (missing authorization).

7 Review WordPress configuration without changing it

When we opened Healthyaa's wp-admin page, the browser was already signed in as user. No password entry was needed. We read settings through the interface and did not click update, activate, deactivate, save, reset or delete controls.

1. Plugins then Installed Plugins: record name, version, active status and update notices.

2. Tools then Site Health: distinguish configuration and performance warnings from exploit findings.

3. Site Health then Info: read WordPress, PHP, environment type and must-use plugin information.

4. Security Optimizer then Login Security and Site Security: record configured protection states.

5. WooCommerce then Settings: inspect Payments, Accounts and Privacy, and Emails.

What we observed on 28 October

The plugin screen showed 49 entries, 35 active, 14 inactive, 23 updates available and auto-updates disabled for all 49. Site Health separately displayed 11 inactive plugins, so the screens did not agree on that count; use the plugin inventory as the stated source and reconcile the difference before cleanup.

Site Health reported WordPress 7.1.3, PHP 8.2.35 and a production environment. Contact Form 7 and Template Kit Import displayed PHP compatibility blockers for offered updates. An update backlog is not a count of exploitable vulnerabilities. Match exact versions, prerequisites and fixes to vendor advisories before assigning vulnerability severity.

SiteGround's administrator/editor MFA toggle was off. Its login-attempt limit was five. System-folder protection, disabling the theme/plugin editor and disabling XML-RPC were enabled. These are configured states; we did not exercise brute-force lockout or test every enforcement path. Other MFA layers were not verified.

Several inactive versions of the custom AI Assistant remained installed. Must-use plugins included Healthyaa Analytics and custom checkout/cart components. Must-use plugins run automatically and deserve source review even though they are separate from the normal activation list.

How to report accurately

Write "SiteGround MFA setting off" rather than "all accounts have no MFA." Write "23 plugin updates offered" rather than "23 vulnerabilities." WordPress Site Health's critical performance labels are not equivalent to a penetration tester's critical security findings.

PHP support reference checked during the assessment: https://www.php.net/supported-versions.php. The PHP 8.2 branch remains security-supported until 31 December 2026; plan migration before that date.

8 Verify anonymous API boundaries

We sent separate requests without the administrator's cookies, using a bounded script with five-second spacing. This tested whether anonymous callers could reach selected protected resources. It did not test every role or every endpoint.

Request path	Observed result
/wp-admin/plugins.php	302 to login
/wp-json/wp/v2/settings	401 rest_forbidden
/wp-json/wp/v2/users?context=edit&per_page=1	401 rest_forbidden_context
/wp-json/wc/v3/settings	401 woocommerce_rest_cannot_view
/wp-json/healthyaa/v1/abandoned-carts	403 healthyaa_rest_forbidden
/xmlrpc.php	403 for GET
/demo-security-review-nonexistent-20261028	404
/wp-json/wctr-api/v1/tyrules/fetch	200 with an empty data array

The refusals are positive results for the specific tested requests. A 200 response from the thank-you rule endpoint returned no rules, so it did not establish sensitive disclosure. The GET refusal on XML-RPC does not prove that all protocol methods were tested.

Repeat one bounded check only if needed

```
curl --silent --show-error --max-time 30 \
  --dump-header settings.headers \
  --output settings.json \
  https://healthyaa.co.uk/wp-json/wp/v2/settings
cat settings.headers
cat settings.json
```

This command contains no login cookie. Expect an access refusal on the current configuration. If a protected resource unexpectedly returns personal information, stop expanding the test and preserve only the minimum evidence needed; do not enumerate more records.

Why discover API routes

The public REST index listed custom abandoned-cart, AI chat and thank-you-rule routes and their supported methods. An index is a map, not proof those operations are open to anonymous callers. We did not submit chat messages or call rule-saving operations.

An initial index read was capped and incomplete. A later bounded request retrieved the complete index. We saved selected route metadata and response shapes rather than customer record contents. The 28 October total was 12 standalone requests, plus separate administrator-browser traffic.

9 Understand payment and email test prerequisites

Stripe was enabled with test mode unchecked. The settings page reported a successful recent live webhook and a pending webhook. That status does not prove that signatures, amounts and replay protection are correctly validated. WooPayments still requested setup.

A SandBox Payment method was also enabled and described as administrator-only. We have not verified its server-side role enforcement. An administrator intentionally using that gateway would demonstrate an allowed test feature, not a customer payment bypass.

Customer notifications were enabled for several order states and account events. FluentSMTP and Post SMTP were both active, with an override warning. Some operational notification recipients were user@example.com. We recorded these observations without sending mail or changing templates.

Prepare isolation before testing checkout

1. Provide a staging URL and a sanitized copy with synthetic customer records. Creating an ordinary production clone alone can copy real data and integrations.

2. Password-protect or otherwise restrict staging. Disable real fulfilment, clinic bookings, production webhooks and external automation there.

3. Use the gateway's test credentials. Do not switch the live store to test mode.

4. Route test mail to an email sink or only the approved inbox. Ensure subjects include demo, and verify the routing with a controlled message before order testing.

5. Create two disposable customers and a dummy product that cannot trigger a real appointment or service.

What the later tests should prove

Scenario	Expected control
Client changes a price or total	Server recalculates from trusted product and discount data.
Payment fails or is cancelled	No paid fulfilment or booking is released.
Payment confirmation is duplicated	Processing remains idempotent; no duplicate fulfilment.
Ordinary customer requests sandbox gateway	Server rejects the method for that role.
Customer A requests B's synthetic order	Server refuses access, including API and download routes.

These tests are pending. No staging URL or verified isolation was supplied. Payment validation must include order state and fulfilment evidence; a thank-you page or order number does not establish that a payment succeeded or was bypassed.

10 Write findings and keep a learning record

A useful finding connects reproducible evidence to a failed control and practical impact. Record passes and limitations as well as concerns. Do not inflate an observation into an exploit or treat an untested area as secure.

Finding template

ID and title:
Target and environment:
Date and tester:
Account role or anonymous context:
Expected behavior:
Exact request or configuration inspected:
Observed status and relevant response:
Impact established:
Uncertainty and limitations:
Remediation:
Evidence filenames:
Retest result:

Worked example from Healthyaa

HYA-01 Administrator MFA configuration. Expected: privileged users have enforced MFA through an identified control. Observed: SiteGround Login Security showed its administrator/editor MFA switch off. Impact: this control does not enforce a second factor if credentials are stolen. Limitation: independent MFA layers were not verified and no account compromise was attempted. Recommendation: establish enforced MFA with recovery procedures, then verify using a disposable privileged account during a planned change window.

Evidence handling

Record method, URL, time, role and response together. Redact passwords, session cookies, API secrets and customer details from screenshots shared outside the assessment. A response hash helps detect later file changes; it does not prove the original observation was correct.

```
shasum -a 256 home.headers home.html > SHA256SUMS.txt
```

The original evidence is in healthyaa-review, with the expanded assessment under healthyaa-review/2026-10-28. The files include public-checks.json, custom-access-checks.json, plugin-inventory.txt and login-controls.png. ASSESSMENT.md records current findings and pending tests.

Learning exercises

1. Explain the difference between 200, 302, 401 and 403 using a captured response.

2. Describe why robots.txt disclosed a lab route but did not cause the authorization flaw.

3. Write one confirmed observation, one passed check and one unanswered question from Healthyaa.

4. For each pending test, name the accounts, synthetic data and expected result needed before running it.

A full engagement ends after scoped tests, evidence review, remediation and retesting. Our completed work is a foundation for that process; checkout, multi-user isolation, persistent input testing, comprehensive advisory matching and recovery validation remain unfinished.

11 Reference material and a reusable test log

These official references explain the tools and exercises used in this guide. Open practice labs in Burp's browser so their traffic appears in HTTP history.

curl command reference

https://curl.se/docs/manpage.html

Chrome Network panel

https://developer.chrome.com/docs/devtools/network/reference/

Burp traffic interception tutorial

https://portswigger.net/burp/documentation/desktop/getting-started/intercepting-http-traffic

Web Security Academy

https://portswigger.net/web-security

Completed lab description

https://portswigger.net/web-security/access-control/lab-unprotected-admin-functionality

Next lab description

https://portswigger.net/web-security/access-control/lab-unprotected-admin-functionality-with-unpredictable-url

PHP supported versions

https://www.php.net/supported-versions.php

Claude-Red repository reviewed

https://github.com/novadot-ORG/Claude-Red

Test log to copy into your own notes

Test	Expected	Observed	Evidence and status
Anonymous settings API	Denied	401	settings.headers and JSON; passed for this request
Admin MFA control	Enforced	Toggle off	Login settings evidence; hardening item
Customer order isolation	A cannot access B	Not tested	Requires staging and two demo accounts

For your internship document, describe which actions you performed manually and which checks the assistant ran. Keep the lab's successful exploit separate from the production review, and include the assessment date whenever quoting a setting or version.